IoTPentBench: A Network-Scale Benchmark for LLM-Based IoT Penetration Testing

Anonymous Authors
Paper #XXXX, Anonymized for Double-Blind Review

***Abstract*—Penetration tests of IoT deployments must reason at the scale of a *network*: dozens of heterogeneous devices, firewall-enforced reachability, and protocol-specific application layers (MQTT, Modbus, LoRaWAN, CoAP). Yet every reproducible benchmark used to evaluate LLM penetration-testing agents to date (AutoPenBench, NYU CTF Bench, Cybench, Vulhub, CAIBench) scores the agent on a *single host* — one VM, one container, one flag. Multi-device reachability, behind-pivot findings, and IoT/OT protocols fall outside this unit of evaluation. We close this gap with IoTPentBench, a reproducible benchmark of 15 IoT scenarios — 10 main scenarios spanning 5 architectural patterns (flat, gateway-centric, segmented IT/OT, hub-star, edge-cloud), 3 scalability stressors (up to 35 devices in a single subnet), and 2 hardened controls (zero injected vulnerabilities, measuring False Positive rate) — across 9 application protocols and 229 vulnerabilities injected deterministically via Ansible with per-vulnerability machine-readable ground truth (OWASP IoT Top 10 and MITRE ATT&CK for ICS mapped). We additionally release MESHSCOUT, a network-native multi-agent harness whose six phases (topology prior, discovery, per-device parallel triage, per-vulnerability verification, intrusion / lateral movement, network-level report) keep LLM context bounded at $O(1)$ per device. Beyond binary flag-capture, IoTPentBench scores findings on a three-level evidence spectrum (L1 detect / L2 exploit / L3 exfiltrate) and on Multi-Hop Reach (MHR$_k$), the recall restricted to ground-truth findings at hop depth $\geq k$ from the entry point. Holding the LLM constant (`MiniMax-M2.7`) to isolate architectural contributions, we compare MESHSCOUT against a non-LLM scanner (Nmap NSE), a single-agent LLM (PentestGPT), a multi-agent LLM without infrastructure graph (CAI, two scope variants), and a multi-agent LLM with task-graph (VulnBot). The evaluation surfaces a structural gap invisible to single-host benchmarks: every baseline collapses to MHR$_k$ $= 0$ for $k \geq 2$ by construction, while the L1/L2/L3 spectrum makes precision and exploitation depth separable rather than conflated. All artifacts (Ansible playbooks, ground truth, evaluator, run logs) are released under an open license.**

***Index Terms*—IoT security, penetration testing, large language models, autonomous agents, benchmark, cyber-physical systems**

1. Introduction

Modern IoT deployments are inherently *networks*: a smart building chains cameras, NVRs, MQTT brokers, access controllers, and HVAC controllers behind a single gateway; an industrial site couples Modbus PLCs and an HMI behind a jump host; a city-scale LoRaWAN deployment routes sensor traffic through edge gateways into cloud APIs. The attack surface that matters to a defender is the union of all reachability paths, not any single host. Real-world incidents (Mirai [25], the Verkada camera breach [26], the Stuxnet lineage in OT [27]) all materialised as multi-hop pivots across this surface, not as one-shot single-host compromises.

The LLM-pentesting literature has matured rapidly, and recent systems report substantial gains on public test corpora (Table 2) — but every one of those corpora (AutoPenBench [17], NYU CTF Bench [18], Cybench [19], CAIBench [21], Vulhub [24]) is a collection of *isolated containers or VMs*, and every agent above is scored on solving one such target at a time. The unit of evaluation is therefore the host, not the network, and the structural properties that distinguish a real IoT deployment from a single VM are invisible to the metric. Behind-pivot vulnerabilities cannot be reached from the perimeter at all, so an agent given only the perimeter IP scores zero on them by construction; IoT- and OT-specific protocols (MQTT, Modbus, LoRaWAN, CoAP) do not appear in HackTheBox or VulnHub corpora and require probes the agent never has to learn; and a binary flag check conflates merely detecting a banner with exploiting it and with exfiltrating the data behind it — three operationally very different outcomes. These gaps are *architectural* (properties of the evaluation surface, not of the agent), so making the agent stronger does not close them. Our contribution closes them with a network-scale benchmark, an evidence-level scoring scheme, and a harness whose phases are designed around the gaps rather than retrofitted to them.

Contributions. We bridge this gap with:

- **IoTPentBench** — the first reproducible benchmark for LLM pentest agents at *network scale*: 5 architectural patterns instantiated by 10 main scenarios, plus 3 scalability stressors (up to 35 devices) and 2 hardened controls (for False Positive rate); 15 scenarios total covering 9 protocols (MQTT, Modbus, LoRaWAN, CoAP, SNMP, SSH, HTTP, FTP,

Telnet) and 229 vulnerabilities deterministically injected via Ansible, with ground truth mapped to OWASP IoT Top 10 and MITRE ATT&CK for ICS.
- **MESHSCOUT** — a network-native harness with six phases (*topology prior* $\rightarrow$ *discovery* $\rightarrow$ *per-device parallel triage* $\rightarrow$ *per-vulnerability verification* $\rightarrow$ *intrusion / lateral movement* $\rightarrow$ *network-level report*), designed to keep LLM context tractable at $/24$ scale.
- **Evidence-level scoring (L1/L2/L3)** — a severity-weighted evaluator that distinguishes detection, exploitation, and data exfiltration, beyond the binary flag-capture of single-host benchmarks.
- **Empirical study** — using a single shared LLM (`MiniMax-M2.7`) to isolate architectural contributions, we compare MESHSCOUT against three categories of baselines: a non-LLM scanner (Nmap NSE), single-agent LLM (PentestGPT), and multi-agent LLM systems with and without graph guidance (CAI in two scope variants, VulnBot with task-graph). We additionally report two ablations of our pipeline that isolate the individual contributions of the infrastructure-graph phase and the multi-hop intrusion phase.

Research questions. RQ1 (overall performance): how well do current LLM-pentest architectures perform at the *network* unit of evaluation, holding the model constant? **RQ2** (evidence depth): how much of an LLM agent's recall stops at mere detection versus reaching exploitation or exfiltration? **RQ3** (architectural sensitivity): which of the five network patterns most degrade LLM performance, and why? **RQ4** (network-scale gap): can existing single-host harnesses, when widened to a CIDR scope, recover the multi-hop reach that motivates this benchmark?

Paper organization. Section 2 surveys LLM pentest agents and benchmarks. Section 3 formalizes the threat model. Section 4 presents IoTPentBench. Section 5 describes the harness architecture. Section 6 and Section 7 present experiments and results. Section 8 and Section 9 discuss limitations and ethics. Artifacts are released at `ANONYMIZED_URL`.

2. Related Work

This section reviews prior work relevant to our approach. We first survey LLM-driven penetration-testing agents and the benchmarks used to evaluate them, then discuss the classical attack-graph literature that informs MESHSCOUT's topology-aware design, and finally review IoT firmware analysis and the security frameworks underlying our scenario taxonomy.

2.1. LLM Penetration-Testing Agents

Happe and Cito [1] reported one of the first peer-reviewed studies of LLM-driven penetration testing, showing that a single LLM driving a shell could autonomously perform privilege-escalation tasks on Linux VMs; the HACKINGBUDDYGPT project [2] extended this single-agent line into an open-source tool. PENTESTGPT [3] introduced the first multi-module design: three coordinated LLM sessions (Reasoning, Generation, Parsing) operate over a *Pentesting Task Tree* (PTT) that encodes the testing state in natural language. Evaluated on 182 hand-curated HackTheBox and VulnHub sub-tasks across 13 targets, it established the recurring failure mode that motivates all subsequent work — loss of session context across long command sequences — and reported a 228.6% task-completion increase over a vanilla GPT-3.5 baseline. AUTOATTACKER [4] (offensive operations) and NEBULA [5] (operator assistant) are subsequent single-LLM variants in the same line as Happe & Cito.

Later systems push specialisation from cognitive roles to penetration phases. VULNBOT [6] decomposes the workflow into three specialised phases (reconnaissance, scanning, exploitation), each driven by five cooperating modules (planner, generator, executor, summariser, memory retriever) over a Penetration Task Graph (PTG) of task dependencies. It reports up to 30.3% task completion on AutoPenBench for its Llama3.1-405B variant, a 21-point gain over the same model used single-agent. AUTOPENTESTER [7] extends the PTT idea with Retrieval-Augmented Generation across five LLM agents (summariser, strategy analyser, generator, results verifier, report generator) plus an agent-computer interface, reporting a +27% subtask completion rate over PENTESTGPT on Hack-The-Box machines. PENTESTAGENT [8] (ASIA CCS'25) and the production-grade CAI [9], co-authored by the PENTESTGPT team, generalise this multi-agent blueprint with modular tool use and a public leaderboard.

A complementary thread improves the underlying agent rather than its orchestration. PENTEST-R1 [10] applies two-stage reinforcement learning on reasoning traces, and XOFFENSE [11] LoRA-fine-tunes Qwen3-32B on offensive corpora (write-ups from TryHackMe / HackTheBox / VulnHub plus WHITERABBITNEO), reaching 79.2% subtask completion on AutoPenBench — outperforming the larger Llama3.1-405B baseline. ENIGMA [12] pairs an LLM with interactive shell tools on CTF challenges, and IRIS [13] couples an LLM with static analysis to surface code-level vulnerabilities. For long-running sessions, CHAP [14] relays distilled context across successive agent instances and AWE [15] adds persistent memory plus browser-backed verification. CHECKMATE [16] couples an LLM agent with a *Classical Planning+* module that encodes preconditions and effects of predefined attack actions, providing explicit causal reasoning that the LLM otherwise lacks; it improves success rates over the strongest LLM-only baseline (Claude Code + Sonnet 4.5) by more than 20% on the Vulhub benchmark.

Despite the diversity of these designs, every system above is evaluated on a *single host*: a HackTheBox machine, a VulnHub VM, or a CTF container. None of them models multi-device reachability as an explicit infrastructure graph or handles IoT-specific application protocols (MQTT, Modbus, LoRaWAN, CoAP).

2.2. LLM Cybersecurity Benchmarks

The dominant academic corpus is AutoPenBench [17], consisting of VulnHub-derived single-VM challenges with scripted flag checkers; it is used by VULNBOT and PENTEST-R1. Recent benchmarks broaden the scope without changing the unit of evaluation: NYU CTF BENCH [18] contributes 200 Jeopardy challenges with reproducible containers, CYBENCH [19] co-evaluates capability and risk on a curated CTF subset, and CYBERSECEVAL [20] measures cybersecurity risk and helpfulness across a large prompt corpus. CAIBENCH [21] consolidates the field into a meta-benchmark spanning Jeopardy CTFs, attack-and-defense ranges, and knowledge tests; its RCTF2 subset adds 27 robotics challenges — the first cyber-physical inclusion in this benchmark family — yet each challenge remains an isolated container.

Two community resources are heavily reused as evaluation substrate by the agents discussed in §2.1 but are not themselves benchmarks in the academic sense. VULHUB [24] ships ∼100+ pre-built Docker images of vulnerable applications and CVE reproductions (one container per CVE); CHECKMATE [16] uses it as its target corpus. AUTOPENBENCH [17] packages 24 in-vitro tasks (access control, web, network, cryptography) plus 12 real-world CVE-based exploits (CVSS 7.5–10.0) with scripted flag checkers, and is the corpus behind VULNBOT, PENTEST-R1, and XOFFENSE. Both resources remain single-container in nature: the unit of evaluation is one image, not a multi-device subnet. Table 2 consolidates the headline numbers each of these systems publishes on its chosen corpus, making explicit that no prior LLM-pentest result is reported on a multi-device IoT network.

A separate community ships reproducible multi-host labs. LUDUS [22] and the GOAD [23] Active-Directory scenario deliver Proxmox+Ansible deployments of representative enterprise networks, methodologically the closest precedent to our infrastructure. They ship no ground-truth schema and no scoring procedure, and have not been used to evaluate LLM agents in the published literature. No prior benchmark therefore offers the combination on which IoTPentBench is built: network-scale evaluation, IoT/OT application protocols (MQTT, Modbus, LoRaWAN, CoAP), per-vulnerability machine-readable ground truth, and evidence-level granularity beyond binary flag capture (Table 1).

2.3. Attack Graphs and Network Reachability

Representing a network as a graph for security analysis predates deep learning by decades. Phillips and Swiler [28] introduced graph-based network-vulnerability analysis, Sheyner *et al.* [29] developed automated attack-graph generation via model checking, and MULVAL [30] cast the problem as Datalog inference, becoming the de facto reference for attack-graph research. These systems compute reachable attacker states from a *static* encoding of host configurations and exposed services. We borrow their reachability abstraction — nodes are devices, edges are firewall-permitted protocol flows — but MESHSCOUT operates on a *live* network: edges are validated by actual probes, the LLM proposes the next action, and the graph is updated as new hosts are discovered. Classical attack-graph generation is therefore complementary rather than competing.

2.4. IoT Security Context

Scenario design and ground-truth taxonomy follow three references: the OWASP IoT Top 10 [31] for vulnerability categories, MITRE ATT&CK for ICS [32] for OT-tactic annotations, and NIST SP 800-82 Rev. 3 [33] for segmentation conventions, notably the IT/OT split underpinning pattern P3 (§4.2). On the empirical side, the closest IoT security work targets firmware rather than deployed networks: Costin *et al.* [34] performed the first large-scale firmware study, FIRMADYNE [35] introduced dynamic firmware emulation for vulnerability discovery, and FIRMAE [36] substantially improved emulation success rates. OWASP IOTGOAT [37] provides a deliberately insecure firmware image used as a teaching artefact. These projects produce vulnerable images; IoTPentBench produces a deployed multi-device network with firewall-enforced reachability, the dimension along which we evaluate MESHSCOUT.

3. Threat Model and Problem Definition

Network-scale pentest. We define *network-scale penetration testing* as the task of: (i) discovering all live hosts in a target IP range, (ii) enumerating services and identifying vulnerabilities on each host, (iii) producing evidence of exploitability where feasible, and (iv) aggregating findings at the network level. The *unit of evaluation* is the network, not an individual host.

Attacker model. The attacker holds network-layer access to the target subnet (post-ingress, pre-compromise). They hold no prior credentials, no privileged vantage, and no knowledge of deployed software beyond what is observable through scans. The attacker has access to standard offensive tooling (Nmap, MQTT clients, SSH, ...) and an LLM with tool-calling. Time and budget are bounded.

Goals.

1) Enumerate all devices reachable from the entry point.
2) For each device, identify vulnerabilities from misconfiguration, weak credentials, information disclosure, insecure protocols, and known CVEs.
3) For each finding, attempt to produce *evidence* at one of three levels: L1 detection (port open, banner leak), L2 exploitation (authenticated session, protocol handshake), L3 data exfiltration (configuration files, credentials, sensor data).
4) Report findings at the network level (severity-ranked, deduplicated).

TABLE 1. Comparison with prior LLM pentest systems and benchmarks. *Scope*: unit of evaluation (single-host = one VM/container; network = multi-device IP range). *Domain*: protocols / target type. *Test corpus*: dataset on which results are reported. Rows are grouped by role: LLM pentest agents (top) and evaluation corpora (bottom).

System	Scope	Domain	Test corpus	IoT-specific	Scoring
LLM pentest agents					
Happe & Cito [1]	single-host	Web/OS	curated Linux VMs	no	flag
PentestGPT [3]	single-host	Web/OS	HTB + VulnHub (13 VMs)	no	sub-task
VulnBot [6]	single-host	Web/OS	AutoPenBench	no	task-graph
AutoPentester [7]	single-host	Web/OS	HackTheBox	no	sub-task
CAI [9]	single-host	Web/CTF	CAIBench	no	CTF
Pentest-R1 [10]	single-host	Web/OS	AutoPenBench	no	sub-task
xOffense [11]	single-host	Web/CTF	AutoPenBench	no	sub-task
EnIGMA [12]	single-host	CTF	NYU CTF Bench	no	flag
CheckMate [16]	single-host	Web/OS	Vulhub	no	sub-task
Evaluation corpora					
AutoPenBench [17]	single-host	Web/OS	24 in-vitro + 12 CVEs	no	flag
NYU CTF Bench [18]	single-host	CTF	200 Jeopardy challenges	no	flag
Cybench [19]	single-host	CTF	curated CTF subset	no	flag
CAIBench / RCTF2 [21]	single-host	mixed	meta-benchmark	27 robotics	CTF
Vulhub [24]	single-host	Web/OS	$\sim$100+ Docker CVEs	no	custom
Ludus + GOAD [22], [23]	multi-host	AD/Win	AD lab (no GT)	no	–
IoTPentBench (ours)	**network**	**MQTT/Modbus/LoRa/CoAP/...**	**15 IoT scenarios (up to 35 dev.), per-vuln YAML**	**yes**	**L1/L2/L3 + MHR$_k$**

TABLE 2. Headline numbers reported by prior LLM pentest systems on their chosen corpus. *Sub-task %* = fraction of predefined intermediate steps solved. *E2E %* = fraction of full target/challenge solved end-to-end. Values are absolute except where prefixed with "+" (relative gain vs the immediately preceding baseline row). Rows are not directly comparable across corpora — the table makes explicit *where* each tool is tested and *what* number it publishes. No prior result is reported on a multi-device IoT network with firewall-enforced reachability, which is the gap IoTPentBench targets.

System	Model	Corpus	Sub-task %	E2E %
GPT-3.5 baseline [3]	GPT-3.5	HTB+VulnHub (13 VMs)	23.1	7.7
GPT-4 baseline [3]	GPT-4	HTB+VulnHub (13 VMs)	52.2	38.5
PentestGPT [3]	GPT-4	HTB+VulnHub (13 VMs)	**66.5**	**46.2**
AutoPentester [7]	GPT-4o	HackTheBox	**+27.0**†	n/r
Llama 3.1-405B baseline [6]	Llama 3.1-405B	AutoPenBench	49.1	9.1
GPT-4o baseline [6]	GPT-4o	AutoPenBench	n/r	21.2
VulnBot [6]	Llama 3.1-405B	AutoPenBench	**69.1**	**30.3**
xOffense [11]	Qwen3-32B (LoRA)	AutoPenBench	**79.2**	n/r
Claude Code (baseline) [16]	Sonnet 4.5	Vulhub	n/r	baseline
CheckMate [16]	Sonnet 4.5 + Plan+	Vulhub	n/r	**+20**†
EnIGMA [12]	GPT-4	NYU CTF (200)	n/r	SOTA
IoTPentBench (ours)	**MiniMax-M2.7**	**15 IoT scenarios (up to /24, 35 dev.)**	**Recall / Prec / F1 / Score / MHR$_k$ / L1L2L3**	

† *relative gain over the immediately preceding baseline row. "n/r" = metric not reported in the source paper.*

Scope and non-goals. We evaluate at the *IP/application layer* on deployed IoT networks. We do *not* evaluate: (a) physical/side-channel attacks on hardware, (b) RF-layer attacks requiring SDR captures, (c) firmware extraction from physical devices, (d) post-exploitation persistence or impact, (e) insider threats or supply-chain compromise. Hardware-assisted attacks are left as future work (§10).

4. IoTPentBench: A Network-Scale Benchmark

4.1. Design principles

IoTPentBench is built around four principles derived from the gap analysis in §2:

Reproducibility. The entire infrastructure deploys from Ansible playbooks against a Proxmox cluster; every vulnerability is injected idempotently with no manual post-deployment steps.

Architectural diversity. Scenarios span five network patterns (§1) with distinct attacker reachability graphs enforced by OpenWrt firewall rules, not just by service labels.

Protocol coverage. Each scenario mixes multiple IoT and OT protocols (MQTT, Modbus, LoRaWAN gateways, CoAP, SNMP, plus standard SSH/HTTP/FTP/Telnet) to exercise protocol-specific reasoning.

Ground-truth granularity. Every injected vulnerability is documented in YAML with `device`, `IP`, `role`, `severity`, `category`, `OWASP IoT` and `MITRE ICS` mappings, optional `CVE`, `indicators` (expected detection strings), and a `verification` command.

Scalability stress Three scenarios (s_11, s_12, s_13) stress network size at 15, 17, and 35 devices respectively, while the 10 main scenarios stay in the 4–8 device range. This separates the pattern-coverage axis from the size axis.

Hardened controls Two scenarios (s_1h, s_4h) re-use the topologies of s_1 and s_4 with all injected vulnerabilities removed. They measure agent False Positive rate on a deployment that should return "no findings".

4.2. Five architectural patterns

Concretely:

1) **P1 — Flat.** All devices are directly reachable from the entry point; no pivot is required.
2) **P2 — Gateway-centric.** A single IoT gateway mediates all access to back-end services; inter-service traffic is blocked without gateway compromise.
3) **P3 — Segmented IT/OT.** Three zones (admin/IT/OT) with access-control lists; the OT zone (Modbus PLCs, HMI) is reachable only via a jump host.
4) **P4 — Hub-centric star.** A central home-automation hub is the only device that can reach MQTT broker, DB, and camera; peripherals cannot talk to each other directly.
5) **P5 — Edge-cloud pivot.** Edge subnet and cloud subnet are separated; only the edge gateway bridges them, making cloud-side services unreachable without edge compromise.

IoTPentBench ships 15 scenarios organised on two axes: 10 *main* scenarios that instantiate the 5 architectural patterns above (with small-to-medium device counts, 4–8 devices each), 3 *scalability* scenarios that stress network size (15, 17, and 35 devices) while re-using patterns P3/P5, and 2 *hardened controls* that ship identical topologies to s_1 and s_4 *without* any injected vulnerability — used to measure agent False Positive rate. The full mapping is given in Table 3.

4.3. Deterministic vulnerability injection

Vulnerabilities are injected by a single Ansible role (`04_inject_vulns.yml`) parameterized by `scenario_id`. Each scenario YAML lists `router_vulns` (OpenWrt-level: Telnet, WAN admin, FTP) and per-service `role` definitions that trigger role-specific injections: *e.g.*, the `mqtt_broker` role sets `allow_anonymous true` and binds on `0.0.0.0:1883`; the `ssh_server` role installs weak credentials (`admin:admin`) and enables `PasswordAuthentication`; the `db_server` role starts MariaDB with a passwordless `root`. Category distribution is shown in §4.

TABLE 3. THE 15 SCENARIOS OF IOTPENTBENCH. *Main* SCENARIOS COVER THE 5 ARCHITECTURAL PATTERNS; *Scalability* SCENARIOS STRESS NETWORK SIZE; *Hardened* CONTROLS SHIP 0 VULNERABILITIES TO MEASURE FALSE POSITIVE RATE.

ID	Pattern / role	Dev.	Vulns	Theme
Main scenarios (pattern coverage)				
`s_1`	P1 Flat	4	12	Minimal IoT (MQTT/Web/SSH)
`s_2`	P1 Flat	6	13	Flat IoT + IT mix
`s_3`	P2 Gateway-centric	8	18	NATO lab replica
`s_4`	P3 Segmented IT/OT	8	18	Industrial (Modbus/HMI)
`s_5`	P4 Hub-centric star	8	15	Smart building (cameras/NVR)
`s_6`	P4 Hub-centric star	6	16	Home automation hub
`s_7`	P5 Edge-cloud pivot	6	14	Edge → cloud API
`s_8`	P3 Segmented IT/OT	8	14	IT/IoT/OT 3-zone industrial
`s_9`	P1 Flat (mesh)	6	11	Mesh IoT smart-city outdoor
`s_10`	P1 Flat (variants)	6	13	Role-variant stress
Scalability stressors				
`s_11`	Smart City (flat)	15	23	3 zones same /24 (no isolation)
`s_13`	VLAN-segmented	17	20	Real Proxmox VLAN bridge (3 VLANs)
`s_12`	Smart City (large)	35	42	Largest network (34 LXC services)
Hardened controls (FP rate)				
`s_1h`	P1 Flat (hardened)	4	0	s_1 with vulns removed
`s_4h`	P3 ICS (hardened)	8	0	s_4 with vulns removed
Total		**145**	**229**	

TABLE 4. VULNERABILITY CATEGORIES AND SEVERITY DISTRIBUTION ACROSS ALL 13 VULNERABILITY-INJECTED SCENARIOS OF IOTPENTBENCH (229 INJECTED INSTANCES; THE 2 HARDENED CONTROLS CONTRIBUTE 0 BY DESIGN). COUNTS DERIVED FROM `GROUND_TRUTH/SCENARIO_*.YAML`.

Category	Count	CRIT	HIGH	MED	LOW
`misconfiguration`	68	7	36	25	0
`no_authentication`	41	17	21	3	0
`data_exposure`	38	0	14	24	0
`default_credentials`	29	3	26	0	0
`info_disclosure`	13	0	0	0	13
`code_injection`	11	10	1	0	0
`cve`	9	0	3	6	0
`insecure_update`	9	0	4	5	0
`weak_crypto`	6	0	0	1	5
`missing_header`	3	0	0	0	3
`privilege_escalation`	2	0	2	0	0
Total	**229**	37	107	64	21

4.4. Ground-truth schema

Each scenario ships a YAML file describing every injected vulnerability. An entry example (from `scenario_1.yaml`):

Each scenario YAML may additionally declare a top-level `bonus_types` list — vulnerability types that are *adjacent* to the injected set (e.g., a hardening recommendation related to the same service) and which, if reported by an agent, are neither counted as True Positives nor as False Positives. This avoids penalising agents for legitimate adjacent findings while keeping the recall denominator fixed. The evaluator leniency this introduces is discussed in §8 (Limitations).

Fig. 1 placeholder: five topology diagrams side-by-side.
(Flat) — (Gateway-centric) — (Segmented IT/OT) — (Hub-star) — (Edge-cloud)

Figure 1. The five architectural patterns in IoTPentBench. Each pattern is enforced by OpenWrt firewall rules that constrain the attacker's reachability from the entry point, not by service labels alone.

```
- id: V1
  device: s1-mqtt
  ip: "192.168.100.11"
  role: mqtt_broker
  title: "MQTT without authentication"
  severity: high
  category: misconfiguration
  owasp_iot: "I1 / I9"
  mitre_ics: "Initial Access, Collection"
  cve: null
  indicators:
    - "allow_anonymous true"
    - "Anonymous clients connected"
  verification: "mosquitto_sub -h 192.168.100.11 -t '#' -v"
```

Figure 2. Ground-truth entry example.

4.5. Evaluator

The evaluator (§ 5.7 integrates results into Phase 6) matches each LLM finding against ground truth in three ordered passes, with each pass consumed at most once per GT entry: (1) exact CVE match; (2) `(ip, vuln_type)` with type canonicalised through the taxonomy (§5.8); (3) `(ip, category)`. Findings that match none of the three passes are counted as False Positives (no severity-only fallback, to avoid spurious matches). Severity weights are $w = \{\text{critical=4}, \text{high=3}, \text{medium=2}, \text{low=1}\}$; loose category matches (pass 3) apply $0.5\times$, severity mismatches within an otherwise valid match apply $0.75\times$. The composite score is the weighted True-Positive sum normalized by $\sum_{g \in \text{GT}} w(g)$.

Severity-aware dedup. When two LLM findings collide on `(ip, type, port)`, the evaluator retains the entry with the *lower* reported severity. This counter-intuitive rule corrects for an empirical bias of LLM agents toward severity inflation (see §7.6): in our pilot calibration runs (3 scenarios, 3 models, ∼N=120 colliding pairs), the lower-severity entry agreed with the injected severity in the majority of cases; we report the exact figure in §7.6.

Evidence levels. Beyond match accuracy, the evaluator tracks, per True Positive, the deepest *evidence level* achieved by the harness: L1 (detection — banner or port), L2 (exploitation — authenticated action), L3 (data exfiltration — sensitive content retrieved). We report L*k*-coverage $= |\{t \in \text{TP} : \text{level}(t) \geq k\}|/|\text{TP}|$. This metric cannot be computed on single-host binary-flag benchmarks and captures a dimension of pentest quality that existing LLM-pentest evaluations conflate.

Hop depth and Multi-Hop Reach. Let $G = (V, E)$ be the firewall-permitted reachability graph (edges are firewall-allowed protocol flows from §5.2) and let $e \in V$ be the attacker entry point (the subnet ingress). For each ground-truth vulnerability g attached to device $d_g \in V$, we define its *hop depth* as the length of the shortest firewall-permitted path:

$$\text{hop\_depth}(g) = \min_{p \in \text{paths}(e \to d_g)} |p|.$$

A flat scenario has $\text{hop\_depth}(g) = 1$ for every device; a gateway-centric scenario has back-end services at $\text{hop\_depth} = 2$ (entry $\to$ gateway $\to$ service); an IT/OT-segmented scenario has the OT zone at $\text{hop\_depth} \geq 3$. We then define the Multi-Hop Reach at depth k as the recall restricted to deep ground-truth findings:

$$\text{MHR}_k = \frac{|\{g \in \text{GT} : \text{hop\_depth}(g) \geq k \wedge g \in \text{TP}\}|}{|\{g \in \text{GT} : \text{hop\_depth}(g) \geq k\}|}.$$

MHR_1 is the raw recall; MHR_2 and MHR_3 are the recall on behind-pivot findings. A single-host agent run on the perimeter IP can only reach hop-depth 1 vulnerabilities, so its MHR_k for $k \geq 2$ is 0 by construction.

5. MeshScout: A Network-Native LLM Pentest Harness

5.1. Overview

The harness decomposes network pentest into six phases with typed deliverables. The key design decision is *parallelization at the device and vulnerability granularity*: rather than a single monolithic agent that must hold the full network state, we spawn dedicated micro-agents per target with constrained tool sets, then aggregate deterministically. This keeps each micro-agent's context bounded *per device* at $O(1)$ in the number of network devices, with cross-device state only re-introduced in Phase 5 (intrusion) and Phase 6 (report).

5.2. Phase 1: Topology prior

Phase 1 loads a graph model of the target network as a directed graph $G = (V, E)$ where V are devices and E are links labeled by protocol. Two modes are supported: (i) *scenario mode* loads the predefined topology from the benchmark YAML; (ii) *discovery mode* starts from an empty graph and instructs Phase 2 to populate V via network scans. The output is a markdown deliverable enumerating attack surface and preliminary pivot candidates (betweenness-centrality ranked).

5.3. Phase 2: Full-network discovery

Phase 2 executes active reconnaissance across the target /24: `nmap_discovery` (host discovery), `nmap` (service/version), `arp_scan` (L2 + vendor), and protocol-specific probes (`mqtt_listen` subscribing to `#` on

Fig. 2 placeholder: 6-phase pipeline diagram.
Phase 1 (topology prior) → Phase 2 (discovery) → Phase 3 (per-device triage) → Phase 4 (per-vuln verification) → Phase 5 (intrusion) → Phase 6 (network report).

Figure 3. MESHSCOUT pipeline. Solid arrows are deliverables; dashed boxes indicate LLM agent calls; Phase 3 and Phase 4 spawn micro-agents in parallel (§5.4–§5.5).

Fig. 3 placeholder: Phase 3 parallelization — n devices → n LLM agents in parallel → deterministic merge.

Figure 4. Per-device parallelization. Each micro-agent receives only its device's scan output, avoiding the context explosion of a monolithic network-level agent.

discovered brokers, `ssh_audit` against SSH servers, `curl_headers` on HTTP). Unlike single-host harnesses, the agent is not given a target IP — it must derive one from the CIDR scope.

5.4. Phase 3: Per-device parallel triage

Phase 3 runs in three sub-phases:

1) **Deterministic scanner pass.** Ansible-driven subprocess wrappers (`ssh_audit`, `curl_headers`, `mqtt_listen`, `modbus_scan`, ...) run in parallel per discovered device and dump structured scan results to disk.
2) **Per-device LLM triage.** For each device, a dedicated LLM agent receives only that device's Phase 2+3a outputs and has access to a minimal tool set: `http_get` (follow-up file retrieval) and `cve_search` (CPE-to-CVE lookup via NVD). The agent produces a JSON vuln list scoped to that device.
3) **Deterministic merge.** Per-device outputs are concatenated, canonicalized through the taxonomy (§5.8), and deduplicated via severity-aware rules.

This decomposition is essential for network scale: a monolithic agent on an 8-device scenario already needs to fit ∼15–40k tokens of scan output into a single context and reason about cross-device relationships simultaneously, and the cost grows linearly with n. Per-device micro-agents flatten this to a constant size, at the cost of losing some cross-device inference capability, which is recovered in Phase 5.

5.5. Phase 4: Per-vulnerability verification with evidence levels

Phase 4 spawns one micro-agent per Phase 3 finding (in parallel, with a concurrency cap). Each micro-agent receives the finding (IP, type, port, details) and full operational tool access: `ssh_login`, `mysql_query`, `mqtt_listen`, `http_get`, `ftp_list`, `modbus_scan`, `telnet_connect`. The agent is prompted to produce the deepest evidence level it can (L1–L3, §4.5) within a bounded turn budget. Outputs include: `status` (CONFIRMED, NOT_EXPLOITABLE, ERROR), `evidence_level`, `command` executed, `stdout` excerpts, and `data_extracted` (when L3).

Findings in CONFIRMED state from Phase 3 override Phase 4 ERROR/FAILED statuses (rationale: Phase 3 is hypothesis generation with directly-observed indicators; Phase 4 crashes should not erase directly-observed findings).

5.6. Phase 5: Intrusion and lateral movement

Phase 5 turns the per-device vulnerability inventory from Phases 3–4 into a network-scoped infiltration campaign. Given the infrastructure graph (Phase 1) and the confirmed credentials/exposed services collected so far, an intrusion agent attempts: (i) *credential spraying* across discovered hosts (reusing harvested SSH/HTTP/MQTT/DB credentials), (ii) *lateral movement* along firewall-permitted edges, with each successful hop adding a new vantage point to the graph, and (iii) *crown-jewel access* on back-end services (databases, NVRs, historian, cloud APIs) that are unreachable from the entry point in segmented patterns (§4.2). The agent records, for every hop, the source node, destination node, edge protocol, and credentials/exploit used. This phase is the sole producer of L_3 evidence and of MHR$_k$ findings at $k \geq 2$; ablating it (Tab. 5, 6) forces those columns to zero by construction.

5.7. Phase 6: Network-level report

Phase 6 consumes all prior deliverables and produces a network-scoped report with: device inventory, per-device vulnerabilities with severity, attack surface summary, multi-hop intrusion narrative (from Phase 5), and actionable recommendations ordered by severity × exposure. This is the only phase with cross-device aggregation reasoning, deliberately placed at the end to avoid bloating earlier phases.

5.8. Taxonomy and tool abstraction

A single module resolves LLM-produced vulnerability type strings into a canonical taxonomy of 11 categories (§4), filtering noise (e.g., meta-observations) and CONFIG-only findings. Tools are defined declaratively as YAML (schema: name, description, JSON-schema parameters, subprocess command template), so adding a new protocol probe requires no Python code.

5.9. Complexity and cost envelope

For a network with n devices and an average of v findings per device, the harness issues $1\,(\text{P1}) + 1\,(\text{P2}) +$

n (P3b) + nv (P4) + 1 (P5) + 1 (P6) = $O(nv)$ LLM agent invocations, each bounded by a fixed turn budget (50 for Phase 1, 50 for Phase 2, 10 per device for Phase 3b, 10 per finding for Phase 4, 30 for Phase 5, 20 for Phase 6). One *call* is one round of (*prompt* → *tool calls* → *response*); one *turn* is one such round inside the budget. Total cost scales linearly with network size, not combinatorially.

6. Experimental Setup

Infrastructure. All scenarios run on a single Proxmox VE host. Networks are Linux bridges (`vmbr1`, `192.168.100.0/24`) with an OpenWrt router as gateway. Per-scenario deployment and teardown are automated via Ansible and complete in under 10 minutes per scenario.

LLM model — fairness lock. To isolate *architectural* contributions from model-specific advantages, all LLM-driven systems compared in this work — our pipeline (and its ablations) and all LLM baselines — run on the *same* general-purpose model: **MiniMax-M2.7** [38]. We deliberately avoid a multi-model matrix: a fair comparison of architectures requires the model variable to be held constant. CAI's specialised `alias1` model (security-fine-tuned, paywalled) is intentionally not used — this study addresses the architecture axis, not the model fine-tuning axis. The choice of MiniMax-M2.7 also keeps the combined API cost at zero under our research subscription, enabling the full k=3-run variance estimation across all systems and scenarios without budget trade-offs.

Baselines. We compare our pipeline against four architectural slots — three LLM-based and one non-LLM:

- **Single-agent LLM** — PENTESTGPT [3] in autonomous mode, the canonical USENIX'24 reference.
- **Multi-agent LLM (no infrastructure graph)** — CAI [9]. We test two variants: *A1* per-IP (one independent CAI session per ground-truth IP, no shared state) and *B* CIDR-scope (a single CAI session given the full `/24`, allowed to discover hosts and pivot via `LinuxCmd` primitives). Both variants receive the same total turn budget as our pipeline.
- **Multi-agent LLM with task-graph** — VULNBOT [6], the most structurally similar competitor. VulnBot's Penetration Task Graph (PTG) is a directed acyclic graph of *tasks* (action dependencies), to be contrasted with our infrastructure graph of *devices* (firewall-enforced reachability). Both are "graph-based", but encode orthogonal abstractions.
- **Non-LLM scanner** — Nmap with all NSE scripts (`--script=default,vuln,auth`) against the same target CIDR. Scanner outputs are converted to the evaluator's JSON schema.

Internal ablations. Two ablations isolate the contribution of our two key design choices: (i) MESHSCOUT–w/o-Phase 1 disables the topology prior (`--phases 2 3 4 5 6`), testing the value of infrastructure-graph guidance; (ii) MESHSCOUT–w/o-Phase 5 disables the intrusion phase (`--phases 1 2 3 4 6`), testing the value of structured multi-hop lateral movement (the only producer of L_3 evidence and of MHR$_k$ findings for $k \geq 2$, §5.6).

Variance. Each (system, scenario) cell is run k=3 times. We report mean, standard deviation, and 95% bootstrap confidence intervals. Pairwise differences (our pipeline vs each baseline) are tested with Mann–Whitney U on F1 and weighted Score per scenario. *Power caveat:* with $n_1 = n_2 = 3$, the two-sided Mann–Whitney U test has a minimum reachable p-value of 0.10, so individual per-scenario comparisons are indicative rather than conclusive. We mitigate this by aggregating across the seven scenarios (combined $n = 21$ per system) for the headline cross-system tests, where power is sufficient. This protocol still exceeds the single-run reporting of PentestGPT [3], AutoPentester [7], AutoPenBench [17], and CAI [9], and matches the multi-run averaging of VulnBot [6] (5-experiment) and Pentest-R1 [10] (pass@3).

Metrics. We report five families of metrics, chosen to (i) align with prior work's expected vocabulary and (ii) surface architectural differences invisible to host-level benchmarks:

- *Task completion* — Recall against the ground-truth vulnerability set, the network-scale analogue of the sub-task / task completion rate reported by PentestGPT, VulnBot, AutoPentester, Pentest-R1, and xOffense.
- *Quality* — Precision, F1, and weighted Score (CVSS-severity-weighted Recall, §4.5). Score plays the role of AutoPentester's "vulnerability coverage" but with severity weights.
- *Multi-Hop Reach* — MHR$_k$ for $k \in \{1, 2, 3\}$, the fraction of ground-truth vulnerabilities at `hop_depth` $\geq k$ that are reached. No prior LLM-pentest benchmark measures network depth explicitly.
- *Evidence depth* — L_1 (detection), L_2 (exploitation), L_3 (exfiltration) coverage; absent from binary flag-capture benchmarks.
- *Cost envelope* — prompt+completion tokens and monetary cost (USD) from provider billing APIs, plus wall-clock time per scenario, matching CAI [9], AutoPentester, and Pentest-R1.

Compute and budget. All runs consume a combined API budget under \$NNN, bounded by per-phase turn caps and the concurrency limit in Phase 4. **[TODO: Insert exact budget after all runs complete (freeze 2 May).]**

7. Evaluation

7.1. Overall performance (RQ1)

[TODO: Write §7.1 prose — headline finding: MESHSCOUT achieves MHR$_2$ ≈XX%, while all baselines collapse to near-zero on multi-hop vulnerabilities. F1 gap to nearest baseline (VulnBot) is ZZ percentage points. Variance across 3 runs stays under YY%.]

TABLE 5. MAIN RESULTS — TASK COMPLETION (RECALL), PRECISION, F1, WEIGHTED SCORE, MULTI-HOP REACH (MHR_k), TOKENS, AND USD COST; AVERAGED OVER THE 10 MAIN SCENARIOS × 3 RUNS. THE 3 SCALABILITY STRESSORS AND 2 HARDENED CONTROLS ARE REPORTED SEPARATELY IN §??. ALL LLM SYSTEMS USE `MiniMax-M2.7` (CF. §6); NMAP NSE IS THE NON-LLM LOWER BOUND. *Task Completion* IS THE NETWORK-SCALE ANALOGUE OF THE SUB-TASK COMPLETION RATE REPORTED BY PENTESTGPT, VULNBOT, AUTOPENTESTER, AND XOFFENSE (§7.1). MHR_k MEASURES THE FRACTION OF GROUND-TRUTH VULNERABILITIES AT `hop_depth` $\geq k$ THAT WERE REACHED; "–" INDICATES NO GT ENTRIES AT THAT DEPTH. CELLS PRE-FILLED WITH 0.00 ARE ARCHITECTURALLY IMPOSSIBLE: SINGLE-HOST LLMS AND SCANNERS CANNOT PIVOT ACROSS FIREWALL BOUNDARIES BY CONSTRUCTION.

	Quality				Multi-Hop Reach			Cost	
System	TC	Prec.	F1	Score	MHR_1	MHR_2	MHR_3	Tokens (M)	USD
MESHSCOUT (full)	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
MESHSCOUT w/o Phase 1 (no infra graph)	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
MESHSCOUT w/o Phase 5 (no intrusion)	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	0.00	0.00	**[TODO:]**	**[TODO:]**
CAI [9] (Variant A1, per-IP)	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	0.00	0.00	**[TODO:]**	**[TODO:]**
CAI [9] (Variant B, CIDR scope)	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
VulnBot [6] (multi-agent + PTG)	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**
PentestGPT [3] (single-agent)	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	0.00	0.00	**[TODO:]**	**[TODO:]**
Nmap NSE (non-LLM scanner)	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	**[TODO:]**	0.00	0.00	–	–

TC = Task Completion (Recall against GT vuln set). Reported across k=3 runs with 95% bootstrap CI and pairwise Mann–Whitney U vs each baseline; this protocol exceeds the single-run reporting of PentestGPT, AutoPentester, AutoPenBench, and CAI, and matches the multi-run averaging of VulnBot and Pentest-R1.

Fig. 4 placeholder: bar chart, F1 and MHR_2 for MESHSCOUT vs CAI (A/B), VulnBot, PentestGPT, Nmap NSE.

Figure 5. Baseline comparison. Scanners capture known CVEs but miss configuration vulnerabilities. Single-host LLM agents (PentestGPT, CAI A1) cannot pivot across firewall boundaries. CAI B (given the full CIDR) and VulnBot (multi-agent + task-graph) approach our pipeline on direct findings but collapse on multi-hop reach (MHR_2, MHR_3).

Fig. 5 placeholder: heatmap scenario × model, cell = weighted Score.

Figure 6. Per-scenario Score heatmap. **[TODO: Note which pattern consistently breaks LLMs.]**

7.2. Network-scale vs baselines (RQ4)

[TODO: Write §7.2 — explicit narrative: (i) scanners win on CVE category, (ii) LLMs win on misconfig/default-creds, (iii) single-host LLMs adapted to network fall behind native harness on F1 by ZZ%.]

7.3. Per-architecture analysis (RQ3)

[TODO: Write §7.3 — pattern-by-pattern commentary. Expected finding: gateway-centric and hub-star degrade most, because they require pivot reasoning. Edge-cloud may also degrade on cloud-side service discovery.]

7.4. Evidence-level depth (RQ2)

[TODO: Write §7.4 — expected: detection (L1) near-parity with scanners, but L2/L3 is LLM-only. This is the strongest argument for LLM agents in this context.]

TABLE 6. EVIDENCE-LEVEL COVERAGE. L_k IS THE FRACTION OF TRUE POSITIVES REACHING LEVEL k OR DEEPER. ABSENT FROM BINARY-FLAG BENCHMARKS.

System	L_1 (detect)	L_2 (exploit)	L_3 (exfiltrate)
MESHSCOUT (full)	**[TODO:]**	**[TODO:]**	**[TODO:]**
MESHSCOUT w/o Phase 5	**[TODO:]**	**[TODO:]**	0.00
CAI (Variant A1) [9]	**[TODO:]**	**[TODO:]**	0.00
CAI (Variant B) [9]	**[TODO:]**	**[TODO:]**	**[TODO:]**
VulnBot [6]	**[TODO:]**	**[TODO:]**	**[TODO:]**
PentestGPT [3]	**[TODO:]**	**[TODO:]**	0.00
Nmap NSE	–	–	–

Fig. 6 placeholder: Pareto plot, USD cost vs weighted Score, one point per system (all on MiniMax-M2.7).

Figure 7. Cost–performance Pareto across systems on MiniMax-M2.7; the multi-model frontier is deferred to future work (§10). **[TODO: Annotate which system dominates on each axis and the efficient frontier.]**

7.5. Per-protocol analysis

[TODO: Write §7.5 — 2 paragraphs. Expected: LLMs excel on MQTT/SSH/HTTP (text-rich, well-represented in training), fall short on Modbus/LoRaWAN (binary, sparse training data). Report numbers per protocol.]

7.6. Failure modes and cost/time envelope

[TODO: Write §7.6 — categorize failure modes: (a) hallucinated findings on devices with no services, (b) missed vulnerabilities on binary protocols, (c) over-reporting info-disclosure without severity calibration. Brief quantitative table.]

8. Discussion and Limitations

Where LLMs help, where they don't. IoTPentBench surfaces an empirical pattern: LLM agents reach high recall on misconfigurations, default credentials, and information disclosure — findings that are textually expressible and well represented in their training corpora. They consistently underperform on binary industrial protocols (Modbus, CoAP, LoRaWAN) and on cross-device inferences that require holding an aggregated network view. This suggests a hybrid deployment model: LLM agents for configuration auditing, traditional scanners for CVE coverage, and human operators for the OT/ICS layer.

Implications for agentic systems. The harness's per-device and per-vulnerability micro-agent structure is one answer to context scaling, but it cedes cross-device inference. Designs that keep a global aggregator agent in the loop without paying the full network-context cost are an open research direction.

Limitations. (i) *Simulated infrastructure.* IoTPentBench scenarios emulate IoT devices on Linux VMs; real constrained hardware introduces embedded-specific attack surfaces (bootloaders, RTOS, firmware) we do not cover. (ii) *Evaluator leniency.* Bonus categories are accepted as non-FP even without ground-truth matches, which inflates precision. (iii) *Model variance.* With k=3 runs per configuration, confidence intervals on F1 remain wide for some scenarios. (iv) *Prompt sensitivity.* We fix a single prompt family across models; per-model prompt tuning would likely raise scores. (v) *Static architectures.* The five patterns are fixed; real deployments evolve over time and include devices not represented here (RFID badge readers, industrial PLCs, medical devices).

9. Ethical Considerations

Isolation. All experiments run on an isolated Proxmox cluster with a dedicated bridge (`vmbr1`) and no route to external networks; the OpenWrt gateway's `WAN` interface is firewall-isolated. No live hosts, production services, or third-party infrastructure are probed.

Responsible disclosure. IoTPentBench vulnerabilities are deterministically injected into our own lab VMs via Ansible. No 0-days are introduced; all categories (misconfig, weak credentials, Terrapin CVE-2023-48795, outdated Dropbear) correspond to patterns already publicly documented. No responsible-disclosure process is triggered.

Misuse risk of released artifact. The benchmark artifact (Ansible playbooks, ground truth, evaluator) does not embed exploitation payloads for unknown vulnerabilities, and the injected weaknesses are common misconfigurations documented in OWASP IoT Top 10. The harness uses only standard pentest tools already in widespread use. We judge the net societal benefit — reproducible evaluation of LLM pentest agents, and a concrete evaluation surface for future defenders — to exceed the incremental misuse risk.

IRB / human subjects. No human subjects are involved. No personal data is collected or processed.

LLM costs and environmental footprint. All reported experiments together consume an estimated **[TODO: X]** kWh and cost under **[TODO: USD Y]** in API billing.

10. Conclusion

We presented IoTPentBench, the first reproducible benchmark for evaluating LLM penetration testing agents at *network scale* on IoT infrastructure, and MeshScout, a network-native multi-agent harness whose infrastructure-graph guidance and dedicated lateral movement phase keep LLM context tractable while explicitly modelling firewall-enforced reachability. Holding the LLM constant (`MiniMax-M2.7`) across our pipeline, two ablations, and five external baselines (CAI A1 per-IP, CAI B CIDR-scope, VulnBot, PentestGPT, Nmap NSE), we isolate the *architectural* contribution: MeshScout outperforms the single-host LLM agents on multi-hop reach (MHR_2, MHR_3) by a margin that collapses near zero for every single-host baseline by construction, and demonstrates that the L1–L3 evidence spectrum exposes architectural differences that binary flag-capture benchmarks conflate.

Future work. (i) *Multi-model robustness* — extending the architectural comparison to recent frontier models (Claude Sonnet 4.6, GPT-5.5, Gemini 3.1 Pro, DeepSeek V4, Qwen 3.6) to confirm model-independence of the architectural advantage; (ii) *Real hardware* — extending to embedded Zigbee, LoRaWAN, and sub-GHz attack surfaces via SDR tooling; (iii) *Defender's side* — using IoTPentBench as a benchmark for LLM-assisted IoT defense (detection, patching recommendation).

All code, scenarios, ground truth, and run logs are released at `ANONYMIZED_URL` under an open license.

References

[1] A. Happe and J. Cito, "Getting pwn'd by AI: Penetration testing with large language models," in *Proc. ACM ESEC/FSE*, 2023.

[2] A. Happe *et al.*, "HackingBuddyGPT: An LLM-driven pentest agent," 2024. [Online]. Available: https://github.com/ipa-lab/hackingBuddyGPT

[3] G. Deng *et al.*, "PentestGPT: Evaluating and harnessing large language models for automated penetration testing," in *Proc. 33rd USENIX Security Symp.*, 2024.

[4] J. Xu *et al.*, "AutoAttacker: A large language model guided system to implement automatic cyber-attacks," *arXiv:2403.01038*, 2024.

[5] O. V. Aguinaga, "Nebula: An LLM-powered penetration testing assistant," *arXiv:2412.00006*, 2024.

[6] H. He *et al.*, "VulnBot: Autonomous penetration testing for a multi-agent collaborative framework," *arXiv:2501.13411*, 2025.

[7] Y. Ginige *et al.*, "AutoPentester: An LLM agent-based framework for automated pentesting," *arXiv:2510.05605*, 2025.

[8] X. Shen *et al.*, "PentestAgent: Incorporating LLM agents to automated penetration testing," in *Proc. 20th ACM Asia Conf. Computer and Communications Security (ASIA CCS)*, 2025. doi: 10.1145/3708821.3733882.

[9] V. Mayoral-Vilches *et al.*, "CAI: An open, bug bounty-ready cybersecurity AI," *arXiv:2504.06017*, 2025.

[10] Anonymous, "Pentest-R1: Towards autonomous penetration testing reasoning optimized via two-stage reinforcement learning," *arXiv:2508.07382*, 2025.

[11] Anonymous, "xOffense: An AI-driven autonomous penetration testing framework with offensive knowledge-enhanced LLMs and multi-agent systems," *arXiv:2509.13021*, 2025.

[12] T. Abramovich *et al.*, "EnIGMA: Interactive tools substantially assist LM agents in solving CTF challenges," in *Proc. ICML*, 2025.

[13] Z. Li, S. Dutta, and M. Naik, "IRIS: LLM-assisted static analysis for detecting security vulnerabilities," in *Proc. NDSS*, 2025.

[14] Anonymous, "Context relay for long-running penetration-testing agents," in *NDSS Workshop on LLM-Assisted Security and Trust Exploration (LAST-X)*, 2026.

[15] Anonymous, "AWE: A memory-augmented multi-agent framework for autonomous web penetration testing," in *NDSS Workshop on LLM-Assisted Security and Trust Exploration (LAST-X)*, 2026.

[16] L. Wang, X. Shi, Z. Li, Y. Jiang, S. Tan, Y. Jiang, J. Cheng, W. Chen, X. Shen, Z. Li, and Y. Chen, "Automated penetration testing with LLM agents and classical planning," *arXiv:2512.11143*, 2025.

[17] Anonymous, "Towards automated penetration testing: Introducing LLM benchmark, analysis, and improvements," *arXiv:2410.17141*, 2024.

[18] M. Shao *et al.*, "NYU CTF Bench: A scalable open-source benchmark dataset for evaluating LLMs in offensive security," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2024.

[19] A. K. Zhang *et al.*, "Cybench: A framework for evaluating cybersecurity capabilities and risk of language models," *arXiv:2408.08926*, 2024.

[20] M. Bhatt *et al.*, "CyberSecEval 2: A wide-ranging cybersecurity evaluation suite for large language models," *arXiv:2404.13161*, 2024.

[21] V. Mayoral-Vilches *et al.*, "CAIBench: A meta-benchmark for evaluating cybersecurity AI agents," *arXiv:2510.24317*, 2025.

[22] Ludus Cloud, "Ludus: Cyber range automation on Proxmox." [Online]. Available: https://ludus.cloud

[23] M3, "GOAD: Game of Active Directory." [Online]. Available: https://github.com/Orange-Cyberdefense/GOAD

[24] Vulhub contributors, "Vulhub: Pre-built vulnerable Docker environments for learning and reproducing CVEs." [Online]. Available: https://github.com/vulhub/vulhub

[25] M. Antonakakis *et al.*, "Understanding the Mirai botnet," in *Proc. 26th USENIX Security Symp.*, 2017.

[26] W. Turton, "Hackers breach thousands of security cameras, exposing Tesla, jails, hospitals," *Bloomberg*, Mar. 2021.

[27] R. Langner, "Stuxnet: Dissecting a cyberwarfare weapon," *IEEE Security & Privacy*, vol. 9, no. 3, pp. 49–51, 2011.

[28] C. Phillips and L. P. Swiler, "A graph-based system for network-vulnerability analysis," in *Proc. New Security Paradigms Workshop*, 1998.

[29] O. Sheyner, J. Haines, S. Jha, R. Lippmann, and J. M. Wing, "Automated generation and analysis of attack graphs," in *Proc. IEEE Symp. Security and Privacy*, 2002.

[30] X. Ou, S. Govindavajhala, and A. W. Appel, "MulVAL: A logic-based network security analyzer," in *Proc. USENIX Security Symp.*, 2005.

[31] OWASP Foundation, "OWASP Internet of Things Top 10," 2024. [Online]. Available: https://owasp.org/www-project-internet-of-things-top-10/

[32] MITRE Corporation, "MITRE ATT&CK for ICS," 2024. [Online]. Available: https://attack.mitre.org/matrices/ics/

[33] K. Stouffer *et al.*, "Guide to operational technology security," National Institute of Standards and Technology, NIST SP 800-82 Rev. 3, 2023.

[34] A. Costin, J. Zaddach, A. Francillon, and D. Balzarotti, "A large-scale analysis of the security of embedded firmwares," in *Proc. USENIX Security Symp.*, 2014.

[35] D. D. Chen, M. Egele, M. Woo, and D. Brumley, "Towards automated dynamic analysis for Linux-based embedded firmware," in *Proc. NDSS*, 2016.

[36] M. Kim, D. Kim, E. Kim, S. Kim, Y. Jang, and Y. Kim, "FirmAE: Towards large-scale emulation of IoT firmware for dynamic analysis," in *Proc. Annual Computer Security Applications Conf. (ACSAC)*, 2020.

[37] OWASP, "IoTGoat: A deliberately insecure IoT firmware," 2024. [Online]. Available: https://github.com/OWASP/IoTGoat

[38] MiniMax, "MiniMax-M2.7: A general-purpose large language model," 2026. [Online]. Available: https://www.minimax.io/models/text/m27